

Temperature and Humidity Controller Design Using FSM and DHT22 Sensors

Prepared by:

Masoumeh Ostvar

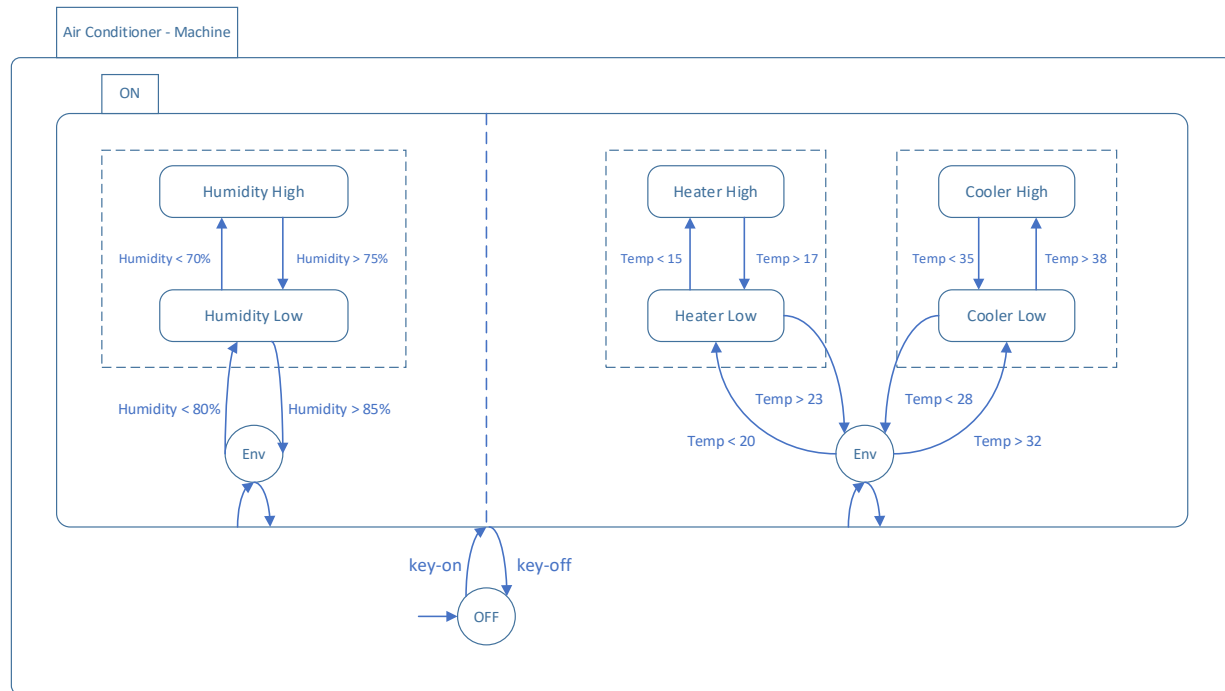
Department of Computer Engineering

Fall 2024

Contents

1. Finite State Machine Diagram for Cooler, Heater, and Humidifier Control.....	2
2. Proteus Simulation Environment	3
3. Implementation Code	3
4. Proteus Simulation Execution Images.....	7

1. Finite State Machine Diagram for Cooler, Heater, and Humidifier Control



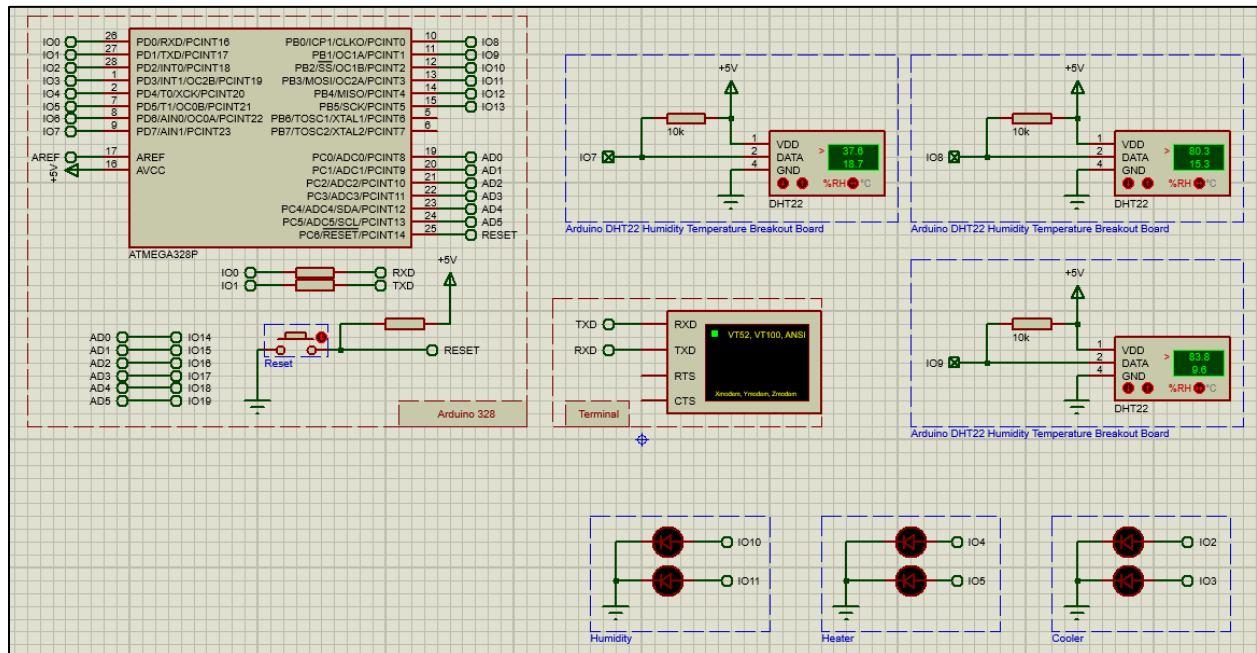
According to the diagram above, the cooler and heater control states operate in parallel with the humidifier control states. When the system starts and exits the OFF state, both sections of the control logic begin operating simultaneously.

As specified in the project requirements, the cooler is activated when the temperature rises above 32°C and initially enters the LOW state. If the temperature increases above 38°C, the cooler transitions to the HIGH state. When the temperature decreases below 35°C, the cooler transitions from HIGH back to LOW. As the temperature continues to decrease below 28°C, the cooler is turned OFF.

As the temperature decreases below 20°C, the heater enters the LOW state. If the temperature continues to decrease below 15°C, the heater transitions to the HIGH state. When the temperature rises above 17°C, the heater transitions from HIGH back to LOW. Finally, when the temperature rises above 23°C, the heater is turned OFF.

Throughout this process, the humidifier continues to operate independently, regardless of whether the system is in a cooling or heating state. When the humidity decreases below 80%, the humidifier enters the LOW state. If the humidity continues to decrease below 70%, it transitions to the HIGH state. When the humidity rises above 75%, the humidifier transitions from HIGH back to LOW. Finally, when the humidity increases above 85%, the humidifier is turned OFF.

2. Proteus Simulation Environment



This environment is designed to control temperature and humidity. Three DHT22 sensors are used to measure temperature and humidity at three different sensing points. Several LEDs are also used to indicate the operating states of the cooler, heater, and humidifier.

3. Implementation Code

First, we define the default pins for the DHT22 sensors and declare the states corresponding to the LOW and HIGH operating modes of the heater, cooler, and humidifier.

```

1  #include "DHT.h"
2  #define DHT0PIN 8
3  #define DHT1PIN 9
4  #define DHT2PIN 4
5  #define DHTTYPE DHT22
6
7  #define cooler_low_level_PIN 2
8  #define cooler_high_level_PIN 3
9
10 #define heater_low_level_PIN 15
11 #define heater_high_level_PIN 16
12
13 #define humidity_low_level_PIN 17
14 #define humidity_high_level_PIN 18
15

```

Three objects of the DHT class are defined, each connected to one of the sensors.

```

16 DHT dht0(DHT0PIN, DHTTYPE,4);
17 DHT dht1(DHT1PIN, DHTTYPE,4);
18 DHT dht2(DHT2PIN, DHTTYPE,4);
19

```

After defining the default settings for the DHT22 sensors, we configure the output pins used to connect the LEDs and initialize the sensors. In this configuration, pins 2 and 3 are assigned to the cooler, pins 15 and 16 to the heater, and pins 17 and 18 to the humidifier.

```
20 void setup(void) {
21     Serial.begin(9600);
22
23     Serial.println("DHT SYSTEM");
24     pinMode(cooler_low_level_PIN,OUTPUT);
25     pinMode(cooler_high_level_PIN,OUTPUT);
26
27     pinMode(heater_low_level_PIN,OUTPUT);
28     pinMode(heater_high_level_PIN,OUTPUT);
29
30     pinMode(humidity_low_level_PIN,OUTPUT);
31     pinMode(humidity_high_level_PIN,OUTPUT);
32
33
34     dht0.begin();
35     dht1.begin();
36     dht2.begin();
37 }
```

In the main loop, we first define variables to store the system states. Then, for each sensor, the readHumidity() and readTemperature() functions are called, and the resulting values are stored in six float variables. Finally, the temperature and humidity readings from the three sensors are averaged to obtain the average environmental values.

```
39 void loop(void) {
40
41     int heater_state = 0;
42     int cooler_state = 0;
43     int humidity_state = 0;
44     int HC_super_state = 0;
45
46     boolean coolerIsOn = false;
47     boolean heaterIsOn = false;
48     boolean humidifier_low = false;
49     boolean humidifier_high = false;
50     boolean cooler_low = false;
51     boolean cooler_high = false;
52     boolean heater_low = false;
53     boolean heater_high = false;
54
55
56
57     while(true){
58         float h0 = dht0.readHumidity();
59         float t0 = dht0.readTemperature();
60         float h1 = dht1.readHumidity();
61         float t1 = dht1.readTemperature();
62         float h2 = dht2.readHumidity();
63         float t2 = dht2.readTemperature();
64         float avgT = (t0+t1+t2)/3;
65         float avgH = (h0+h1+h2)/3;
```

In this section, the humidity level is monitored, and the humidifier state is determined based on the average humidity. If the average humidity falls below 80%, the humidifier starts operating and transitions to the LOW state. If the humidifier is in the LOW state and the humidity decreases below 70%, it transitions to the HIGH state. If the humidity rises above 85%, the humidifier returns to the default state and is turned OFF. If the humidifier is in the HIGH state and the average humidity rises above 75%, it transitions back to the LOW state.

```

67 //check humidity
68
69 if(avgH>85 && humidity_state == 1){
70     humidity_state = 0;
71     humidifier_low = 0;
72     humidifier_high = 0;
73 }
74
75 if(avgH<70 && humidity_state==1){
76     humidity_state = 2;
77     humidifier_low = 0;
78     humidifier_high = 1;
79 }
80
81
82 if(avgH<80 && humidity_state ==0){
83     humidity_state = 1;
84     humidifier_low = 1;
85     humidifier_high = 0;
86 }
87
88 if(avgH>75 && humidity_state == 2){
89     humidity_state = 1;
90     humidifier_low = 1;
91     humidifier_high = 0;
92 }
93

```

In this section, the temperature level is monitored, and the states of the cooler and heater are determined based on the average temperature. For the heater, if the ambient temperature falls below 20°C, the heater is turned ON and enters the LOW state. If the ambient temperature decreases below 15°C, the heater transitions from LOW to HIGH. When the heater is in the HIGH state, if the ambient temperature rises above 17°C, it transitions back to the LOW state. Finally, if the average temperature rises above 23°C, the system returns to the default state and the heater is turned OFF.

```

94 //check cooler & heater
95
96 if(HC_super_state == 0 || heaterIsOn){
97
98     if(avgT<20 && heater_state == 0){
99         HC_super_state = 2;
100         heaterIsOn = true;
101         heater_state = 1;
102         heater_low = 1;
103         heater_high = 0;
104     }
105
106     if(avgT<15 && heater_state == 1){
107         HC_super_state = 2;
108         heaterIsOn = true;
109         heater_state = 2;
110         heater_low = 0;
111         heater_high = 1;
112     }
113
114     if(avgT>17 && heater_state == 2){
115         HC_super_state = 2;
116         heaterIsOn = true;
117         heater_state = 1;
118         heater_low = 1;
119         heater_high = 0;
120     }
121
122     if(avgT > 23 && heater_state == 1){
123         heaterIsOn = false;
124         heater_state = 0;
125         heater_low = 0;
126         heater_high = 0;
127         HC_super_state = 0;
128     }
129 }

```

For the cooler, if the ambient temperature rises above 32°C, the cooler is turned ON and enters the LOW state. If the ambient temperature increases above 38°C, the cooler transitions to the HIGH state. However, if the temperature decreases below 35°C, the cooler transitions back to the LOW state. Finally, if the temperature falls below 28°C, the system exits the cooler control states and returns to the default state, turning the cooler OFF.

```

133 if(HC_super_state == 0 || coolerIsOn){
134
135     if(avgT > 32 && cooler_state==0){
136         HC_super_state = 1;
137         coolerIsOn = true;
138         cooler_state = 1;
139         cooler_low = 1;
140         cooler_high = 0;
141     }
142
143     if(avgT>38 && cooler_state==1){
144         HC_super_state = 1;
145         coolerIsOn = true;
146         cooler_state = 2;
147         cooler_low = 0;
148         cooler_high = 1;
149     }
150
151     if(avgT<35 && cooler_state==2){
152         HC_super_state = 1;
153         coolerIsOn = true;
154         cooler_state = 1;
155         cooler_low = 1;
156         cooler_high = 0;
157     }
158
159     if(avgT < 28 && cooler_state == 1){
160         coolerIsOn = false;
161         cooler_state = 0;
162         cooler_low = 0;
163         cooler_high = 0;
164         HC_super_state = 0;
165     }
166 }

```

In this section, the LED states are updated based on the calculated system states.

```

167
168     digitalWrite(heater_low_level_PIN,heater_low);
169     digitalWrite(heater_high_level_PIN,heater_high);
170     digitalWrite(cooler_low_level_PIN,cooler_low);
171     digitalWrite(cooler_high_level_PIN,cooler_high);
172     digitalWrite(humidity_low_level_PIN,humidifier_low);
173     digitalWrite(humidity_high_level_PIN,humidifier_high);
174

```

In the following section, if there is an issue with the sensor pins or the temperature and humidity readings cannot be obtained, the corresponding error messages are output from the sensors.

```

175 // Check if any reads failed and exit early (to try again).
176 if (isnan(h0) || isnan(t0)) {
177     Serial.println("Failed to read from DHT0 sensor!");
178     return;
179 }
180 if (isnan(h1) || isnan(t1)) {
181     Serial.println("Failed to read from DHT1 sensor!");
182     return;
183 }
184
185 if (isnan(h2) || isnan(t2)) {
186     Serial.println("Failed to read from DHT2 sensor!");
187     return;
188 }

```

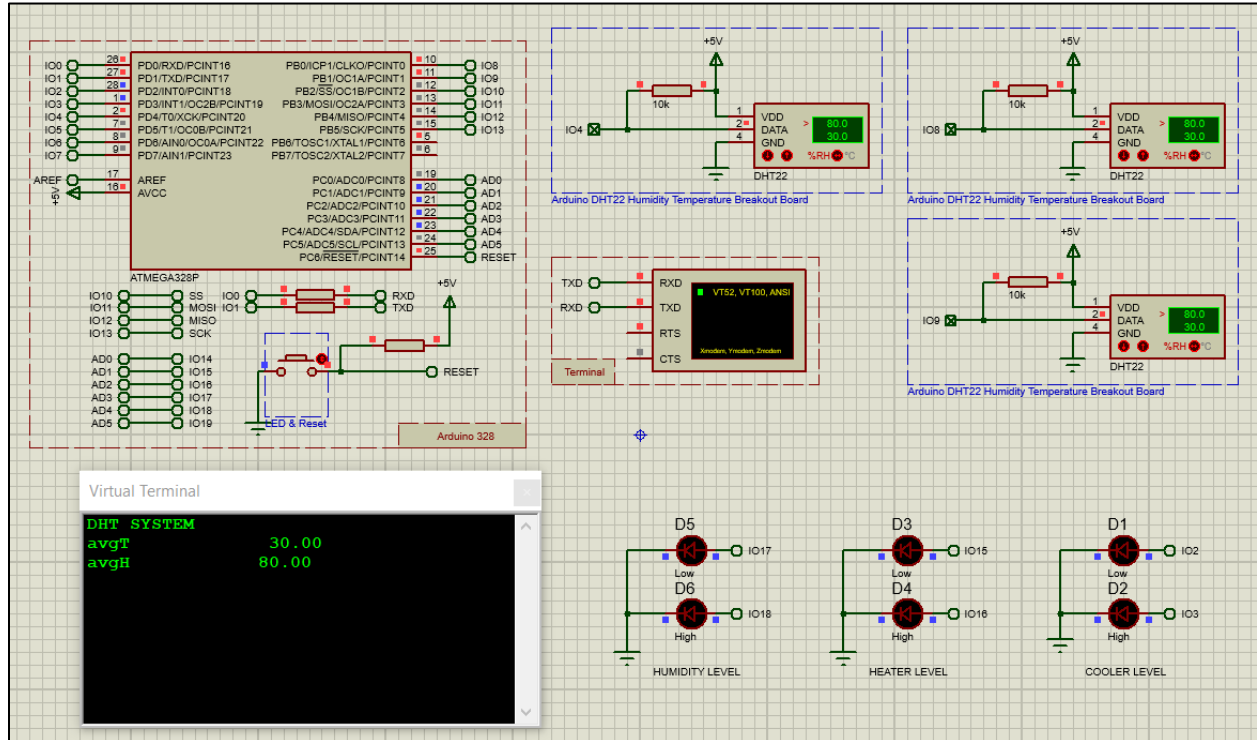
Finally, the readings obtained from all three sensors are averaged, and the average temperature and humidity values are displayed on the terminal. In this section, the display format for the terminal output is also specified. Using the delay() function, the sensor sampling and state update process is performed every 1.5 seconds.

```

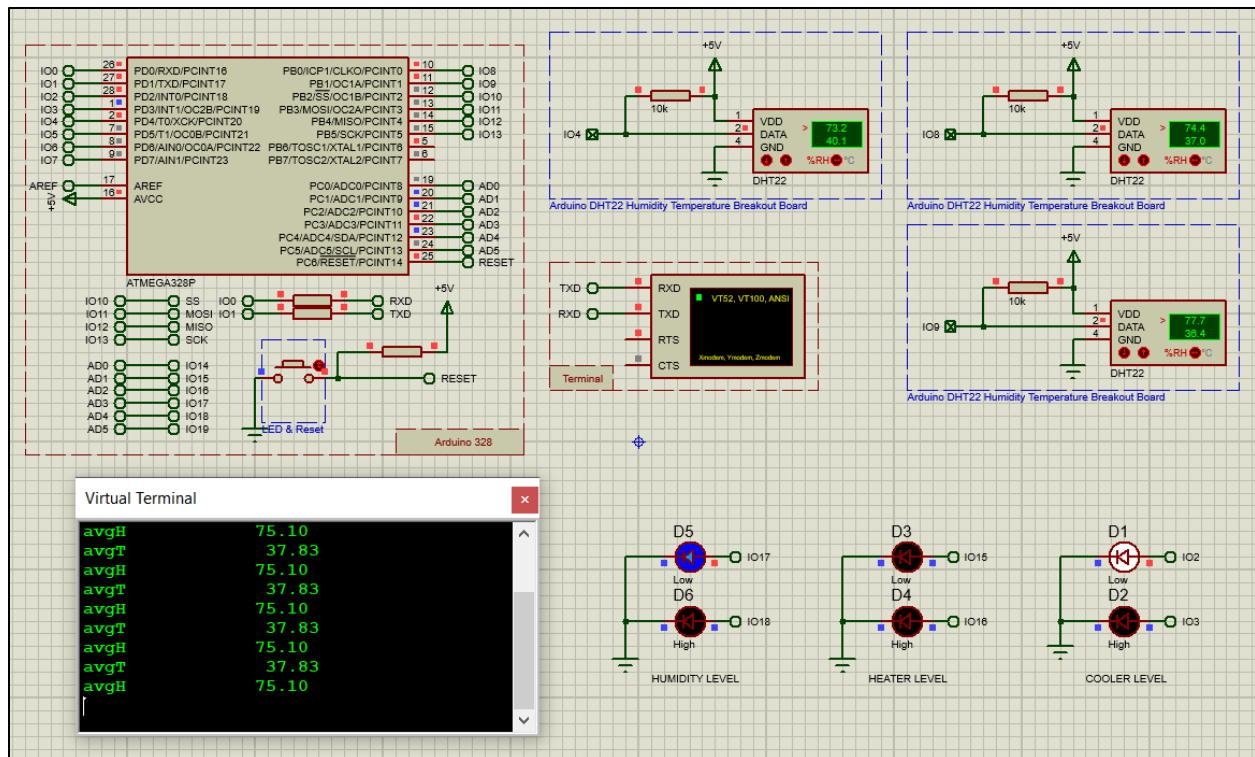
190 Serial.print("avgT      ");
191 Serial.print(avgT);
192 Serial.println();
193 Serial.print("avgH      ");
194 Serial.print(avgH);
195 Serial.println();
196
197 delay(1500);
198     }
199 }

```

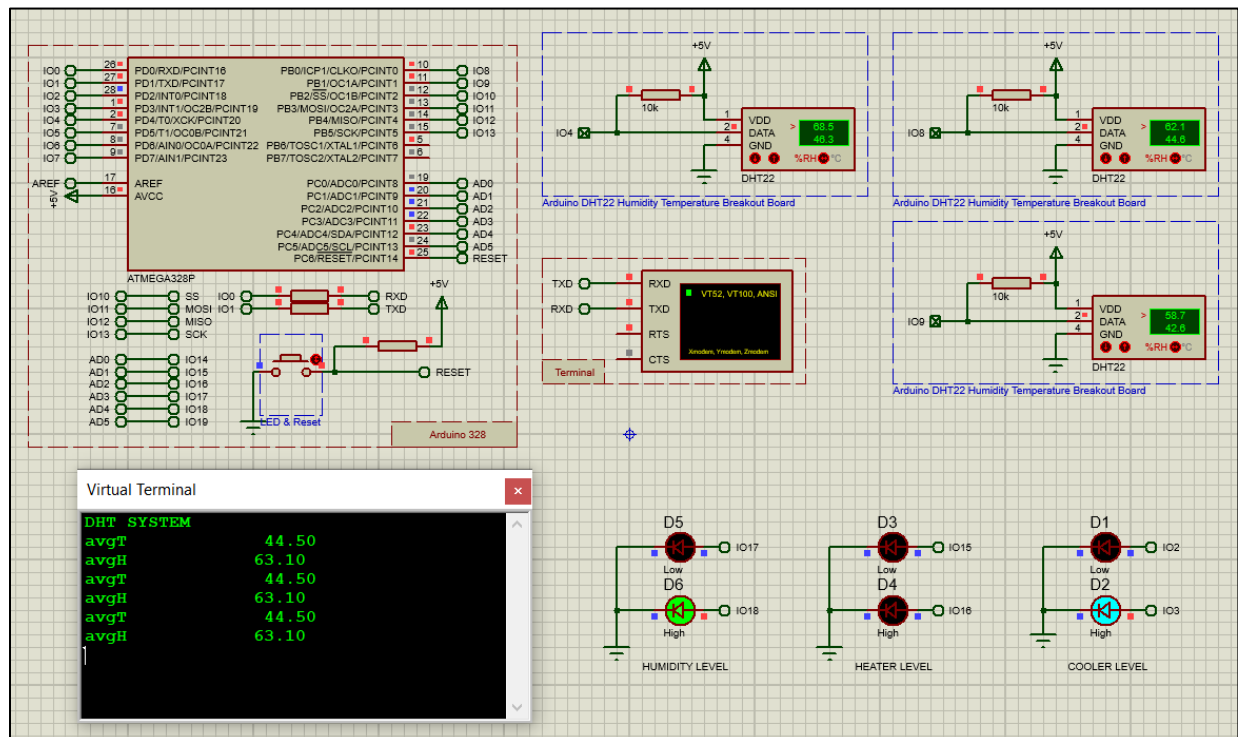
4. Proteus Simulation Execution Images



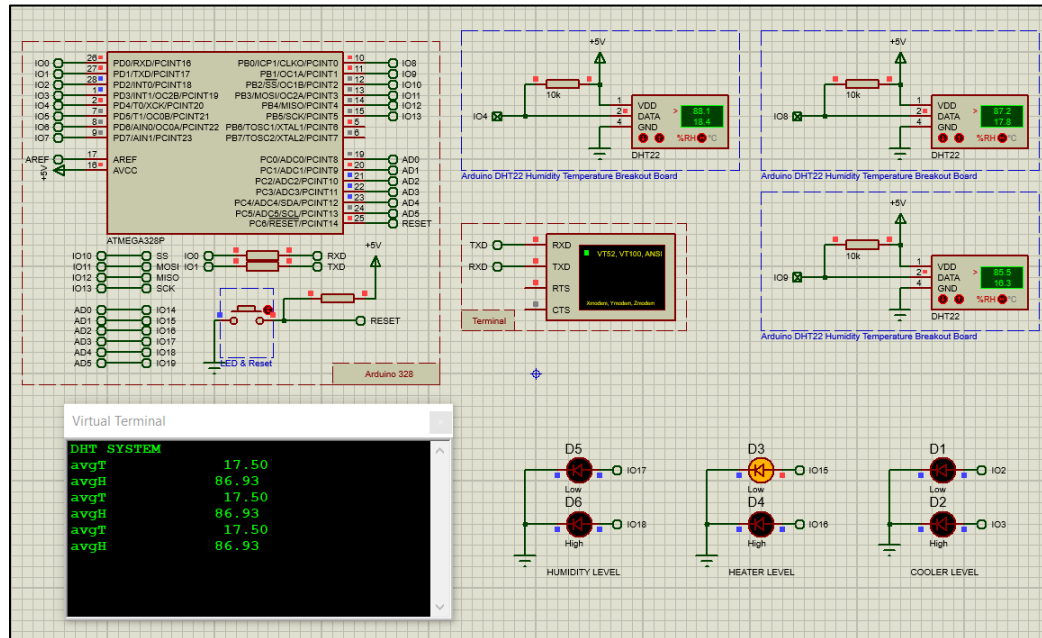
In the initial state, the average ambient temperature is 30°C and the humidity level is above 80%. Therefore, none of the devices is turned on, and all three controllers remain in their default states. As a result, none of the LEDs is illuminated.



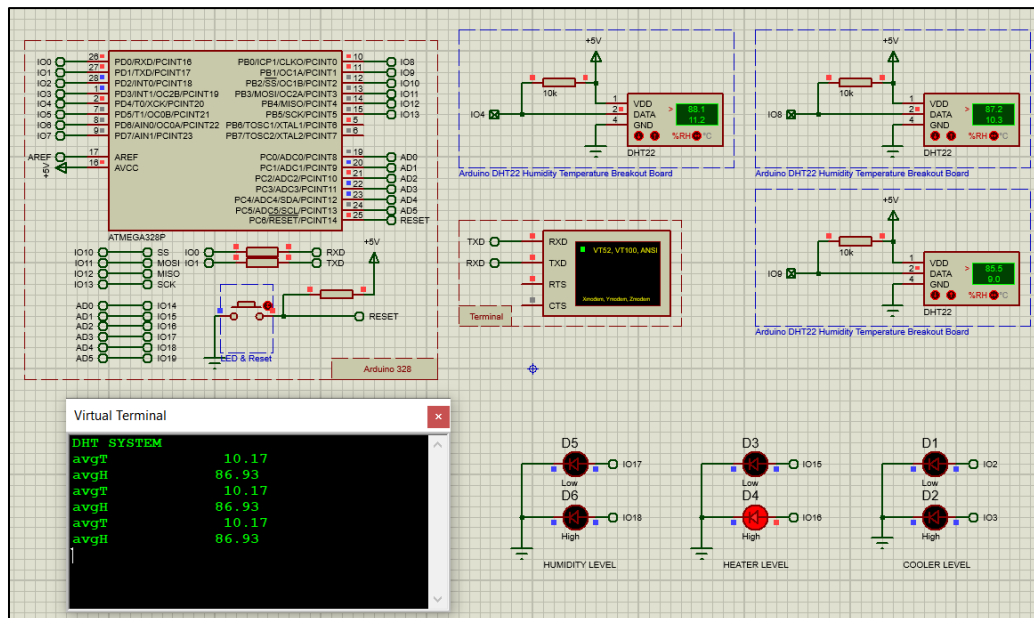
As the temperature increases, the average temperature reaches 37.83°C, which is above 32°C. Therefore, the cooler is activated in the LOW state, and the white LED turns on. At the same time, as the average humidity decreases to 75.10%, which is below 80%, the humidifier is activated in the LOW state, and the blue LED also turns on.



As the temperature increases, the average temperature reaches 44.50°C , which is above 38°C . Therefore, the cooler is activated in the HIGH state, and the cyan LED turns on. At the same time, as the average humidity decreases to 63.10% , which is below 70% , the humidifier is activated in the HIGH state, and the green LED also turns on.



As the temperature decreases, the average temperature reaches 17.50°C , which is below 20°C . Therefore, the heater is activated in the LOW state, and the orange LED turns on.



As the temperature decreases, the average temperature reaches 10.17°C , which is below 15°C . Therefore, the heater is activated in the HIGH state, and the red LED turns on.